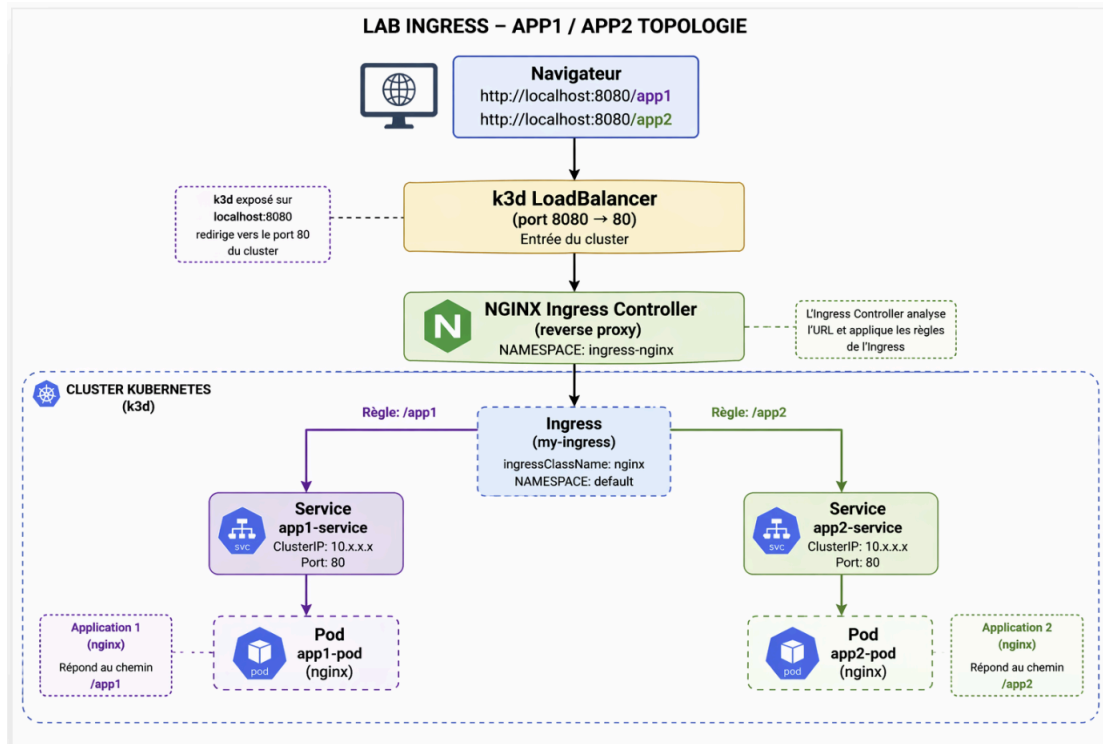


Lab Kubernetes Ingress (k3d)

Table of Contents

Lab Kubernetes Ingress (k3d)	1
1 Objectif	2
2 Topologie	2
 3. Créer le cluster avec Ingress activé	3
 4 Installer un Ingress Controller	3
 5 Déployer deux applications nginx	4
5.1 app1.yaml	4
app1.yaml :	4
5.2 app2.yaml	4
5.3 Appliquer :	5
5.4 Verifier et lister	5
 6. Créer l'Ingress	6
6.1 classe d'ingress	6
6.2 ingress.yaml	6
6.3 Appliquer :	7
6.4 Vérifier	7
 7. Tester	8
 8. Récapitulatifs	9
9 Nettoyage du cluster	9
10 : Annexes – schémas ingress	10



1 Objectif

- Déployer **2 apps web (nginx)**
- Exposer via **Ingress**
- Accéder via des routes :
 - /app1
 - /app2

2 Topologie

```
Navigateur
↓
localhost:8080
↓
k3d LoadBalancer
↓
Ingress Controller (Traefik)
↓
Ingress rules (/app1 ou /app2)
↓
Service
↓
Pod (nginx)
```

3. Créer le cluster avec Ingress activé

Avec k3d, il faut exposer le port 80 du load balancer :

Mettre dans le fichier cluster.sh la commande suivante :

```
k3d cluster create ingress-lab \
-p "8080:80@loadbalancer"
```

👉 On accèderas ensuite via :
<http://localhost:8080>

```
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$ k3d cluster create ingress-lab -p "8080:80@loadbalancer"
INFO[0000] portmapping '8080:80' targets the loadbalancer: defaulting to [servers*:proxy agents*:proxy]
INFO[0000] Prep: Network
INFO[0000] Created network 'k3d-ingress-lab'
INFO[0000] Created image volume k3d-ingress-lab-images
INFO[0000] Starting new tools node...
INFO[0000] Starting node 'k3d-ingress-lab-tools'
INFO[0001] Creating node 'k3d-ingress-lab-server-0'
INFO[0001] Creating LoadBalancer 'k3d-ingress-lab-serverlb'
INFO[0001] Using the k3d-tools node to gather environment information
INFO[0001] Starting new tools node...
INFO[0002] Starting node 'k3d-ingress-lab-tools'
INFO[0003] Starting cluster 'ingress-lab'
INFO[0003] Starting servers...
INFO[0003] Starting node 'k3d-ingress-lab-server-0'
INFO[0008] All agents already running.
INFO[0008] Starting helpers...
INFO[0008] Starting node 'k3d-ingress-lab-serverlb'
INFO[0014] Injecting records for hostAliases (incl. host.k3d.internal) and for 3 network members into CoreDNS configmap.
..
INFO[0017] Cluster 'ingress-lab' created successfully!
INFO[0017] You can now use it like this:
kubectrl cluster-info
henri@Henri:~/labs/kcna-ingress-labs$
```

4 Installer un Ingress Controller

Kubernetes **n'a pas d'Ingress actif par défaut** → il faut en installer un.

On utilise le plus classique :

👉 NGINX Ingress Controller

```
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider
/cloud/deploy.yaml
```

Attendre que tout soit prêt :

```
kubectl get pods -n ingress-nginx
```

5 Déployer deux applications nginx

5.1 app1.yaml

Créer un fichier app1.yaml

Qui contient le code ci-dessous :

app1.yaml :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app1
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app1
  template:
    metadata:
      labels:
        app: app1
    spec:
      containers:
        - name: nginx
          image: nginx

# Service :

apiVersion: v1
kind: Service
metadata:
  name: app1-service
spec:
  selector:
    app: app1
  ports:
```

```
- port: 80
  targetPort: 80
```

5.2 [app2.yaml](#)

Même chose :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app2
spec:
  replicas: 1
  selector:
    matchLabels:
      app: app2
  template:
    metadata:
      labels:
        app: app2
    spec:
      containers:
        - name: nginx
          image: nginx
```

Service :

```
apiVersion: v1
kind: Service
metadata:
  name: app2-service
spec:
  selector:
    app: app2
  ports:
    - port: 80
      targetPort: 80
```

5.3 [Appliquer :](#)

kubect1 create deployment app1 --image=nginx

kubect1 create deployment app2 --image=nginx

5.4 [Verifier et lister](#)

Kubectl get deployments

```
henri@Henri:~/labs/kcna-ingress-labs$  
henri@Henri:~/labs/kcna-ingress-labs$ kubectl get deployments  
NAME      READY   UP-TO-DATE   AVAILABLE   AGE  
app1      1/1     1            1           18m  
app2      1/1     1            1           14m  
henri@Henri:~/labs/kcna-ingress-labs$  
henri@Henri:~/labs/kcna-ingress-labs$
```

Kubectl get pods

```
henri@Henri:~/labs/kcna-ingress-labs$  
henri@Henri:~/labs/kcna-ingress-labs$ kubectl get pods  
NAME                                READY   STATUS    RESTARTS   AGE  
app1-6b9c98df97-n8h7s              1/1     Running   0          4m22s  
app2-558b7bd854-ff65v              1/1     Running   0          45s  
henri@Henri:~/labs/kcna-ingress-labs$  
henri@Henri:~/labs/kcna-ingress-labs$
```

Lister les services actifs du cluster :

kubectl get svc
ou bien

kubectl get services

```
henri@Henri:~/labs/kcna-ingress-labs$  
henri@Henri:~/labs/kcna-ingress-labs$ kubectl get svc  
NAME             TYPE          CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE  
app1-service     ClusterIP     10.43.223.105   <none>        80/TCP           2m4s  
app2-service     ClusterIP     10.43.241.90    <none>        80/TCP           68s  
kubernetes       ClusterIP     10.43.0.1       <none>        443/TCP          56m  
henri@Henri:~/labs/kcna-ingress-labs$
```

6. Créer l'Ingress

6.1 classe d'ingress

Vérifier les type d'ingress disponibles :

```
kubectl get ingressclass
```

Dans notre cas K3d utilise l'ingress : traefik (cela peut varier selon le type de cluster ou l'outil utilisé pour . k3d ou minikube ou kubeadm

```
NAME      CONTROLLER  
traefik   traefik.io/ingress-controller
```

il faudra donc modifier dans le fichier ingress.yaml la valeur du champ : ingressClassName par la valeur : traefik , il faut utiliser l'ingress qui est affiché par la commande : kubectl get ingress ou kubectl describe ingressclass

6.2 ingress.yaml

Créer un fichier : ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  ingressClassName: traefik
  rules:
  - http:
      paths:
      - path: /app1
        pathType: Prefix
        backend:
          service:
            name: app1-service
            port:
              number: 80
      - path: /app2
        pathType: Prefix
        backend:
          service:
            name: app2-service
            port:
              number: 80
```

6.3 Appliquer :

kubectl apply -f ingress.yaml

```
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$ kubectl apply -f ingress.yaml
ingress.networking.k8s.io/my-ingress created
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$
```

6.4 Vérifier

kubectl get ingress

```

henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$ kubectl get ingress
NAME          CLASS    HOSTS      ADDRESS      PORTS      AGE
my-ingress    traefik  *          172.18.0.3   80         48m
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$

```

Commande pur vérifier la composition du ingress :

```
kubectl describe ingress my-ingress
```

```

henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$ kubectl describe ingress my-ingress
Name:          my-ingress
Labels:         <none>
Namespace:     default
Address:        172.18.0.3
Ingress Class:  traefik
Default backend: <default>
Rules:
  Host      Path  Backends
  ----      -
  *          /app1  app1-service:80 (10.42.0.9:80)
            /app2  app2-service:80 (10.42.0.10:80)
Annotations:  <none>
Events:       <none>
henri@Henri:~/labs/kcna-ingress-labs$

```

Commande pour vérifier les endpoints :

```
kubectl get endpoints
```

```

henri@Henri:~/labs/kcna-ingress-labs$
henri@Henri:~/labs/kcna-ingress-labs$ kubectl get endpoints
NAME          ENDPOINTS          AGE
app1-service  10.42.0.9:80       41m
app2-service  10.42.0.10:80      40m
kubernetes    172.18.0.3:6443    95m
henri@Henri:~/labs/kcna-ingress-labs$

```

kubectl get pods --show-labels

Dans ton navigateur :

<http://localhost:8080/app1>

<http://localhost:8080/app2>

ou bien :

curl <http://localhost:8080/app1>

curl <http://localhost:8080/app2>

```
henri@Henri:~/labs/kcna-ingress-labs$ curl http://localhost:8080/app1
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.29.8</center>
</body>
</html>
henri@Henri:~/labs/kcna-ingress-labs$
```

```
henri@Henri:~/labs/kcna-ingress-labs$ curl http://localhost:8080/app2
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.29.8</center>
</body>
</html>
henri@Henri:~/labs/kcna-ingress-labs$
```

8. Récapitulatifs

👉 Ingress = **reverse proxy HTTP** dans Kubernetes

👉 Il permet :

- routing par **URL**
- routing par **hostname**
- centralisation de l'accès

👉 Différence clé :

- **Service** → expose un pod
- **Ingress** → route vers plusieurs services

9 Nettoyage du cluster

```
henri@Henri:~/labs/kcna-ingress-labs$ kubectl delete ingress --all
kubectl delete svc --all
kubectl delete deployment --all
ingress.networking.k8s.io "my-ingress" deleted from default namespace
service "app1-service" deleted from default namespace
service "app2-service" deleted from default namespace
service "kubernetes" deleted from default namespace
deployment.apps "app1" deleted from default namespace
deployment.apps "app2" deleted from default namespace
henri@Henri:~/labs/kcna-ingress-labs$ kubectl delete all --all
service "kubernetes" deleted from default namespace
henri@Henri:~/labs/kcna-ingress-labs$
```

10 : Annexes – schémas ingress

Ci-joint quelques schémas ingress , classiques

